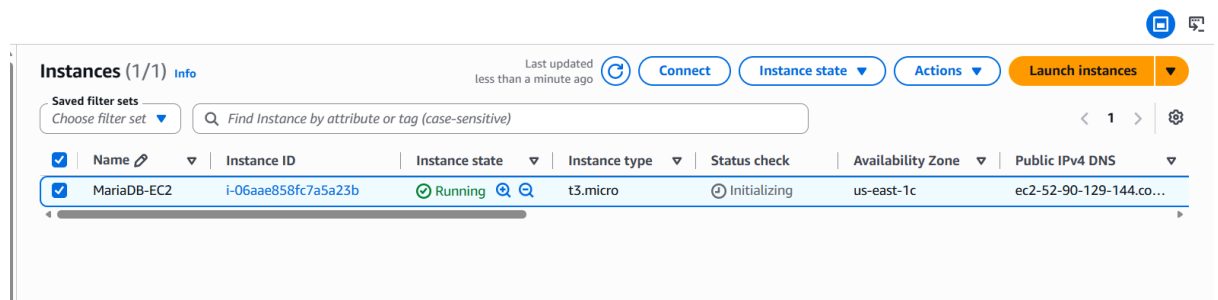


AWS RDS TASK

1. Create MariaDB DB on EC2.

Step 1: Launch an EC2 Instance

- Go to **EC2**.
- Click **Launch Instance**.
- Name: MariaDB-EC2
- AMI: **Amazon Linux 2023** (or Amazon Linux 2)
- Instance Type: t2.micro
- Create a Key Pair (or use an existing one).
- Security Group:
 - SSH (22) → Your IP
- Launch the instance.



Step 2: Connect to EC2

```
ssh -i key.pem ec2-user@<Public-IP>
```

Become the root user:

```
sudo su -
```

```
SRIHARI@LAPTOP-N3TTSS9I MINGW64 ~/Downloads  
$ ssh -i "devops.key.pem" ec2-user@ec2-52-90-129-144.compute-1.amazonaws.com  
The authenticity of host 'ec2-52-90-129-144.compute-1.amazonaws.com (52.90.129.144)' can't be established.  
ED25519 key fingerprint is SHA256:NobYF2XlHpqXO5qpvroXYBpw/Wy4MNd4U84hkakVI3o.  
This key is not known by any other names  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
Warning: Permanently added 'ec2-52-90-129-144.compute-1.amazonaws.com' (ED25519)  
to the list of known hosts.
```

The screenshot shows a terminal window titled 'SRIHARI@LAPTOP-N3TTSS9I'. The user has executed an SSH command to connect to an AWS EC2 instance named 'ec2-52-90-129-144'. The terminal displays standard SSH warning messages about host fingerprints and asks for confirmation to proceed. A custom ASCII art logo consisting of wavy horizontal lines forming a stylized shape is visible on the left side of the screen. Below the logo, the system identifies itself as 'Amazon Linux 2023' and provides the URL '<https://aws.amazon.com/linux/amazon-linux-2023>'. At the bottom, the prompt changes from '[ec2-user@ip-172-31-3-14 ~]' to '[root@ip-172-31-3-14 ec2-user]#', indicating successful login with root privileges via sudo.

```
#_~\#####  
~~~~ \#####\  
~~~~ \|###|  
~~~~ \|#/ https://aws.amazon.com/linux/amazon-linux-2023  
~~~~ V~'|'->  
~~~~ .-. /  
~~~~ _/_'_/_/_'  
[ec2-user@ip-172-31-3-14 ~]$ sudo su  
[root@ip-172-31-3-14 ec2-user]# |
```

Step 3: Install MariaDB

Amazon Linux 2:

```
yum update -y
```

```
yum install mariadb-server -y
```

Amazon Linux 2023:

```
dnf install mariadb105-server -y
```

```
[root@ip-172-31-3-14 ec2-user]# yum update -y
Amazon Linux 2023 Kernel Livepatch repository
Last metadata expiration check: 0:00:01 ago on Mon Aug 3 12:56:21 2026.
Dependencies resolved.
Nothing to do.
Complete!
[root@ip-172-31-3-14 ec2-user]# yum install mariadb-server -y
Last metadata expiration check: 0:00:10 ago on Mon Aug 3 12:56:21 2026.
No match for argument: mariadb-server
Error: Unable to find a match: mariadb-server
[root@ip-172-31-3-14 ec2-user]# dnf install mariadb105-server -y
Last metadata expiration check: 0:00:35 ago on Mon Aug 3 12:56:21 2026.
Dependencies resolved.
```

Package	Architecture	Version
Installing:		
mariadb105-server	x86_64	3:10.5.29-1.amzn2023.0.1
Installing dependencies:		
mariadb-connector-c	x86_64	3.3.10-1.amzn2023.0.2
mariadb-connector-c-config	noarch	3.3.10-1.amzn2023.0.2
mariadb105	x86_64	3:10.5.29-1.amzn2023.0.1
mariadb105-common	x86_64	3:10.5.29-1.amzn2023.0.1
mariadb105-errmsg	x86_64	3:10.5.29-1.amzn2023.0.1
mysql-selinux	noarch	1.0.14-2.amzn2023.0.1

Step 4: Start MariaDB

```
systemctl start mariadb
```

```
systemctl enable mariadb
```

```
systemctl status mariadb
```

You should see:

active (running)

```
[root@ip-172-31-3-14 ec2-user]# systemctl start mariadb
[root@ip-172-31-3-14 ec2-user]# systemctl enable mariadb
Created symlink /etc/systemd/system/mysql.service → /usr/lib/systemd/system/mariadb.service.
Created symlink /etc/systemd/system/mysqld.service → /usr/lib/systemd/system/mariadb.service.
Created symlink /etc/systemd/system/multi-user.target.wants/mariadb.service → /usr/lib/systemd/system/mariadb.service.
[root@ip-172-31-3-14 ec2-user]# systemctl status mariadb
● mariadb.service - MariaDB 10.5 database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; preset: disabled)
   Active: active (running) since Mon 2026-08-03 12:59:02 UTC; 50s ago
     Docs: man:mariadb(8)
           https://mariadb.com/kb/en/library/systemd/
   Main PID: 27364 (mariabdd)
   Status: "Taking your SQL requests now..."
     Tasks: 13 (limit: 1059)
    Memory: 66.2M
       CPU: 557ms
    CGroup: /system.slice/mariadb.service
            └─27364 /usr/libexec/mariabdd --basedir=/usr

Aug 03 12:59:02 ip-172-31-3-14.ec2.internal mariadb-prepare-db-dir[27319]: The second is mysql@localhost, it has no password either
Aug 03 12:59:02 ip-172-31-3-14.ec2.internal mariadb-prepare-db-dir[27319]: you need to be the system 'mysql' user to connect.
Aug 03 12:59:02 ip-172-31-3-14.ec2.internal mariadb-prepare-db-dir[27319]: After connecting you can set the password, if you would
Aug 03 12:59:02 ip-172-31-3-14.ec2.internal mariadb-prepare-db-dir[27319]: able to connect as any of these users with a password and
Aug 03 12:59:02 ip-172-31-3-14.ec2.internal mariadb-prepare-db-dir[27319]: See the MariaDB Knowledgebase at https://mariadb.com/kb
```

Step 5: Set Root Password

```
mysqladmin -u root password "admin123456"
```

Step 6: Login to MariaDB

```
mysql -u root -p
```

Enter:

```
admin123456
```

Step 7: Create Database

```
CREATE DATABASE ec2db;
```

Check:

```
SHOW DATABASES;
```

Step 8: Use Database

```
USE ec2db;
```

Step 9: Create Table

```
CREATE TABLE table1
```

```
(
```

```
id INT,
```

```
name VARCHAR(50)
```

```
);
```

```
[root@ip-172-31-3-14 ec2-user]# mysqladmin -u root password "admin123456"
[root@ip-172-31-3-14 ec2-user]# mysql -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 4
Server version: 10.5.29-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> CREATE DATABASE ec2db;
Query OK, 1 row affected (0.001 sec)

MariaDB [(none)]> SHOW DATABASES;
+-----+
| Database |
+-----+
| ec2db    |
| information_schema |
| mysql    |
| performance_schema |
+-----+
4 rows in set (0.000 sec)

MariaDB [(none)]> USE ec2db;
Database changed
MariaDB [ec2db]> CREATE TABLE table1
-> (
-> id INT,
-> name VARCHAR(50)
-> );
Query OK, 0 rows affected (0.009 sec)

MariaDB [ec2db]>
```

2. Insert some dummy data.

Step 10: Insert Data

```
INSERT INTO table1 VALUES
```

```
(1,'Virat'),
```

```
(2,'Sachin'),
```

```
(3,'Dhoni'),
```

```
(4,'ABD');
```

Step 11: Verify

```
SELECT * FROM table1;
```

Expected Output:

id	name
1	Virat
2	Sachin
3	Dhoni
4	ABD

Exit:

exit

```
MariaDB [ec2db]> INSERT INTO table1 VALUES
-> (1,'Virat'),
-> (2,'Sachin'),
-> (3,'Dhoni'),
-> (4,'ABD');
Query OK, 4 rows affected (0.002 sec)
Records: 4  Duplicates: 0  Warnings: 0

MariaDB [ec2db]> SELECT * FROM table1;
+-----+-----+
| id | name |
+-----+-----+
| 1 | Virat |
| 2 | Sachin |
| 3 | Dhoni |
| 4 | ABD |
+-----+-----+
4 rows in set (0.000 sec)

MariaDB [ec2db]> exit
Bye
[root@ip-172-31-3-14 ec2-user]#
```

3. Take the backup of dummy data on EC2.

Take Backup

Run:

```
mysqldump -u root -p ec2db > ec2db.sql
```

Verify:

ls

You should see:

ec2db.sql

```
[root@ip-172-31-3-14 ec2-user]# mysqldump -u root -p ec2db > ec2db.sql
Enter password:
[root@ip-172-31-3-14 ec2-user]# ls
ec2db.sql
[root@ip-172-31-3-14 ec2-user]#
```

4. Launch MariaDB RDS instance.

Step 1

Go to

AWS Console → RDS

Click

Create Database

Step 2

Choose

Standard Create

Step 3

Engine

MariaDB

Step 4

Template

Free Tier

Step 5

DB Identifier

mariadb-rds

Step 6

Master Username

rdsuser

Password

admin123456

Step 7

Instance

db.t3.micro

Step 8

Storage

20 GB

Step 9

Connectivity

Choose:

- Same VPC as EC2
 - Public Access = Yes (for lab purposes)
 - Create a new security group or use an existing one
-

Step 10

Create Database

Wait until the status becomes

Available

The screenshot displays the AWS Management Console interface for the 'Databases' section. At the top, a green notification banner states 'Successfully created database database-rds' with a 'View connection details' link. Below this, the 'Databases (1)' section shows a table with one entry. The table has columns for DB identifier, Status, Role, Engine, Upgrade rollout order, Region, and Size. The entry 'database-rds' is shown with a status of 'Available', role of 'Instance', engine of 'MariaDB', and size of 'db.t4g.micro'. The console also includes a search bar, a 'Group resources' toggle, and buttons for 'Modify', 'Actions', and 'Create database'.

DB identifier	Status	Role	Engine	Upgrade rollout order	Region	Size
database-rds	Available	Instance	MariaDB	SECOND	us-east-1a	db.t4g.micro

5. Migrate database from EC2 to RDS.

Open the **RDS Security Group**.

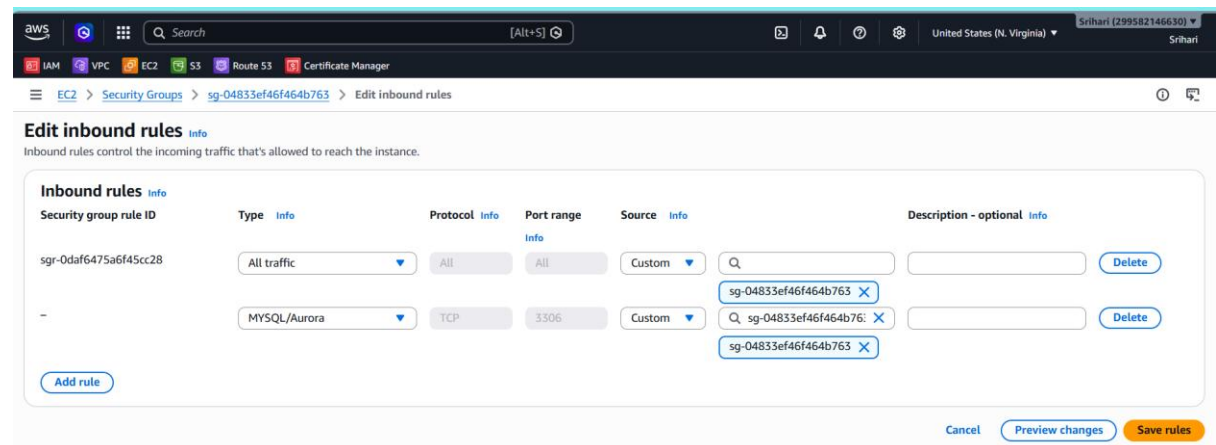
Add an inbound rule:

Type : MySQL/Aurora

Port : 3306

Source : EC2 Security Group

Save the rule.



Lab 5: Migrate Database

Find the **RDS Endpoint**.

Example:

database-rds.c2jeigsek2ps.us-east-1.rds.amazonaws.com

Run:

```
mysql \
```

```
-h database-rds.c2jeigsek2ps.us-east-1.rds.amazonaws.com \
```

```
-u admin \
```

```
-p
```

Create the database:

```
CREATE DATABASE rdsdb;
```

```
exit
```

Import the backup:

```
mysql \
```

```
-h database-rds.c2jeigsek2ps.us-east-1.rds.amazonaws.com \
```

```
-u aadmin \  
-p rdsdb < ec2db.sql
```

Lab 6: Verify

Login:

mysql \

-h mariadb-rds.abc123.eu-north-1.rds.amazonaws.com \

-u rdsuser \

-p

Run:

USE rdsdb;

SELECT * FROM table1;

You should see:

Virat

Sachin

Dhoni

ABD

```
[root@ip-172-31-3-14 ec2-user]# wget https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem  
--2026-08-03 15:20:59-- https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem  
Resolving truststore.pki.rds.amazonaws.com (truststore.pki.rds.amazonaws.com)... 13.226.209.61, 13.226.209.110, ...  
Connecting to truststore.pki.rds.amazonaws.com (truststore.pki.rds.amazonaws.com)[13.226.209.61]:443... connected.  
HTTP request sent, awaiting response... 200 OK  
Length: 165408 (162K) [binary/octet-stream]  
Saving to: 'global-bundle.pem'  
global-bundle.pem 100%[=====] 161.53K  
2026-08-03 15:20:59 (46.0 MB/s) - 'global-bundle.pem' saved [165408/165408]  
[root@ip-172-31-3-14 ec2-user]# mysql -h database-rds.c2jeigsek2ps.us-east-1.rds.amazonaws.com -P 3306 -u admin -p --ssl-verify-server-cert --ssl-ca=./global-bundle.pem  
Enter password:  
^C  
[root@ip-172-31-3-14 ec2-user]# ls  
ec2db_backup.sql global-bundle.pem  
[root@ip-172-31-3-14 ec2-user]# mysql -h database-rds.c2jeigsek2ps.us-east-1.rds.amazonaws.com \  
> -u admin \  
> -p \  
> --ssl-ca=global-bundle.pem  
Enter password:  
ERROR 2002 (HY000): Can't connect to MySQL server on 'database-rds.c2jeigsek2ps.us-east-1.rds.amazonaws.com' (115)  
[root@ip-172-31-3-14 ec2-user]# mysql -h database-rds.c2jeigsek2ps.us-east-1.rds.amazonaws.com -u admin -p --ssl-ca=global-bundle.pem  
Enter password:  
Welcome to the MariaDB monitor. Commands end with ; or \g.  
Your MariaDB connection id is 54  
Server version: 11.8.8-MariaDB-log managed by https://aws.amazon.com/rds/  
Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.  
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.  
MariaDB [(none)]> show databases;  
+-----+  
| Database |  
+-----+  
| information_schema |  
| innodb |  
| mysql |  
| performance_schema |  
| sys |  
+-----+  
5 rows in set (0.006 sec)
```



```

MariaDB [(none)]> create database
-> ec2db;
Query OK, 1 row affected (0.011 sec)

MariaDB [(none)]> show databases;
+-----+
| Database |
+-----+
| ec2db    |
| information_schema |
| innodb   |
| mysql    |
| performance_schema |
| sys      |
+-----+
6 rows in set (0.001 sec)

MariaDB [(none)]> exit
Bye
[root@ip-172-31-3-14 ec2-user]# mysql -h database-rds.c2jeigsek2ps.us-east-1.rds.amazonaws.com -P 3306 -u admin -p ec2bd < ec2bd_backup.sql
Enter password:
ERROR 1049 (42000): Unknown database 'ec2bd'
[root@ip-172-31-3-14 ec2-user]# mysql -h database-rds.c2jeigsek2ps.us-east-1.rds.amazonaws.com -P 3306 -u admin -p ec2bd < ec2bd_backup.sql
Enter password:
ERROR 1045 (28000): Access denied for user 'admin'@'172.31.3.14' (using password: YES)
[root@ip-172-31-3-14 ec2-user]# mysql -h database-rds.c2jeigsek2ps.us-east-1.rds.amazonaws.com -P 3306 -u admin -p ec2bd < ec2bd_backup.sql
bash: ec2bd_backup.sql: No such file or directory
[root@ip-172-31-3-14 ec2-user]# ll
total 168
-rw-r--r-- 1 root root 1963 Aug 3 15:09 ec2bd_backup.sql
-rw-r--r-- 1 root root 163408 Jul 24 2025 global-bundle.pem
[root@ip-172-31-3-14 ec2-user]# mysql -h database-rds.c2jeigsek2ps.us-east-1.rds.amazonaws.com -P 3306 -u admin -p ec2bd < ec2bd_backup.sql
Enter password:
[root@ip-172-31-3-14 ec2-user]# mysql -h database-rds.c2jeigsek2ps.us-east-1.rds.amazonaws.com -P 3306 -u admin -p -e "select * from ec2db.table1;"
Enter password:
+----+-----+
| id | name |
+----+-----+
| 1  | Virat |
| 2  | Sachin |
| 3  | Dhoni |
| 4  | ABD   |
+----+-----+
[root@ip-172-31-3-14 ec2-user]#

```

6. Install MySQL DB on EC2.

Step 1: Update the system

```
sudo dnf update -y
```

Step 2: Install the MySQL Community Repository

```
sudo dnf install -y https://dev.mysql.com/get/mysql84-community-release-el9-1.noarch.rpm
```

```

[root@ip-172-31-3-14 ec2-user]# sudo dnf install -y https://dev.mysql.com/get/mysql84-community-release-el9-1.noarch.rpm
MySQL Connectors Community
Errors during downloading metadata for repository 'mysql-connectors-community':
 - Status code: 404 for https://repo.mysql.com/yum/mysql-connectors-community/el/2023.12.20260727/x86_64/repodata/repomd.xml (IP: 23.
Error: Failed to download metadata for repo 'mysql-connectors-community': Cannot download repomd.xml: Cannot download repodata/repomd.
MySQL 8.4 LTS Community Server
Errors during downloading metadata for repository 'mysql-8.4-lts-community':
 - Status code: 404 for https://repo.mysql.com/yum/mysql-8.4-community/el/2023.12.20260727/x86_64/repodata/repomd.xml (IP: 23.33.203.
Error: Failed to download metadata for repo 'mysql-8.4-lts-community': Cannot download repomd.xml: Cannot download repodata/repomd.xml
MySQL Tools 8.4 LTS Community
Errors during downloading metadata for repository 'mysql-tools-8.4-lts-community':
 - Status code: 404 for https://repo.mysql.com/yum/mysql-tools-8.4-community/el/2023.12.20260727/x86_64/repodata/repomd.xml (IP: 23.3
Error: Failed to download metadata for repo 'mysql-tools-8.4-lts-community': Cannot download repomd.xml: Cannot download repodata/repod
Ignoring repositories: mysql-connectors-community, mysql-8.4-lts-community, mysql-tools-8.4-lts-community
Last metadata expiration check: 2:52:57 ago on Mon Aug 3 12:56:21 2026.
mysql84-community-release-el9-1.noarch.rpm
Dependencies resolved.

```

Package	Architecture	Version
Downgrading:		
mysql84-community-release	noarch	el9-1

```

Transaction Summary

```

Step 3: Disable MySQL 8.4 repository

```
sudo dnf config-manager --disable mysql-8.4-lts-community
```

Step 4: Enable MySQL 8.0 repository

```
sudo dnf config-manager --enable mysql80-community
```

Step 5: Install MySQL Server

```
sudo dnf install -y mysql-community-server
```

Step 6: Start MySQL Service

```
sudo systemctl enable mysqld
```

```
sudo systemctl start mysqld
```

Step 7: Check MySQL Status

```
sudo systemctl status mysqld
```

Expected output:

Active: active (running)

Note: If any error comes:

Check this

Step 1: Check the MySQL service status

Run:

```
sudo systemctl status mysqld.service
```

Step 2: Check the MySQL logs

Run:

```
sudo journalctl -xeu mysqld.service
```

or

```
sudo cat /var/log/mysqld.log
```

Since you previously had MariaDB installed, the failure is often caused by leftover MariaDB data.

Run:

```
ls -ld /var/lib/mysql
```

If the directory exists, rename it:

```
sudo mv /var/lib/mysql /var/lib/mysql_backup
```

Then initialize MySQL:

```
sudo mysqld --initialize --user=mysql
```

Now start MySQL again:

```
sudo systemctl start mysqld
```

Check its status:

```
sudo systemctl status mysqld
```

```
[root@ip-172-31-3-14 ec2-user]# sudo dnf config-manager --disable mysql-8.4-lts-community
[root@ip-172-31-3-14 ec2-user]# sudo dnf config-manager --enable mysql80-community
[root@ip-172-31-3-14 ec2-user]# sudo dnf config-manager --enable mysql80-community
[root@ip-172-31-3-14 ec2-user]# sudo dnf install -y mysql-community-server
MySQL 8.0 Community Server
MySQL Connectors Community
MySQL Tools 8.4 LTS Community
Dependencies resolved.
=====
Package                                         Architecture      Version
=====
Installing:
mysql-community-server                         x86_64             8.0.46-1.el9
Installing dependencies:
mysql-community-client                         x86_64             8.0.46-1.el9
mysql-community-client-plugins                x86_64             8.0.46-1.el9
mysql-community-common                        x86_64             8.0.46-1.el9
mysql-community-icu-data-files                x86_64             8.0.46-1.el9
mysql-community-libs                          x86_64             8.0.46-1.el9
=====
Transaction Summary
=====
Install 6 Packages

[root@ip-172-31-3-14 ec2-user]# ls -ld /var/lib/mysql
drwxr-x--x. 6 mysql mysql 16384 Aug  3 15:50 /var/lib/mysql
[root@ip-172-31-3-14 ec2-user]# sudo mv /var/lib/mysql /var/lib/mysql_backup
[root@ip-172-31-3-14 ec2-user]# sudo mysqld --initialize --user=mysql
[root@ip-172-31-3-14 ec2-user]# sudo systemctl start mysqld
[root@ip-172-31-3-14 ec2-user]# sudo systemctl status mysqld
● mysqld.service - MySQL Server
   Loaded: loaded (/usr/lib/systemd/system/mysqld.service; enabled; preset: disabled)
   Active: active (running) since Mon 2026-08-03 15:51:52 UTC; 5s ago
     Docs: man:mysqld(8)
           http://dev.mysql.com/doc/refman/en/using-systemd.html
  Process: 34109 ExecStartPre=/usr/bin/mysqld_pre_systemd (code=exited, status=0/SUCCESS)
    Main PID: 34136 (mysqld)
      Status: "Server is operational"
     Tasks: 38 (limit: 1059)
    Memory: 387.5M
       CPU: 1.565s
    CGroup: /system.slice/mysqld.service
            └─34136 /usr/sbin/mysqld

Aug 03 15:51:50 ip-172-31-3-14.ec2.internal systemd[1]: Starting mysqld.service - MySQL Server...
Aug 03 15:51:52 ip-172-31-3-14.ec2.internal systemd[1]: Started mysqld.service - MySQL Server.
[root@ip-172-31-3-14 ec2-user]#
```

7. Launch MySQL RDS image.

Step 1: Open RDS

1. Login to the AWS Console.
2. Search for **RDS**.
3. Click **Amazon RDS**.

Step 2: Create Database

Click **Create database**.

Step 3: Choose Creation Method

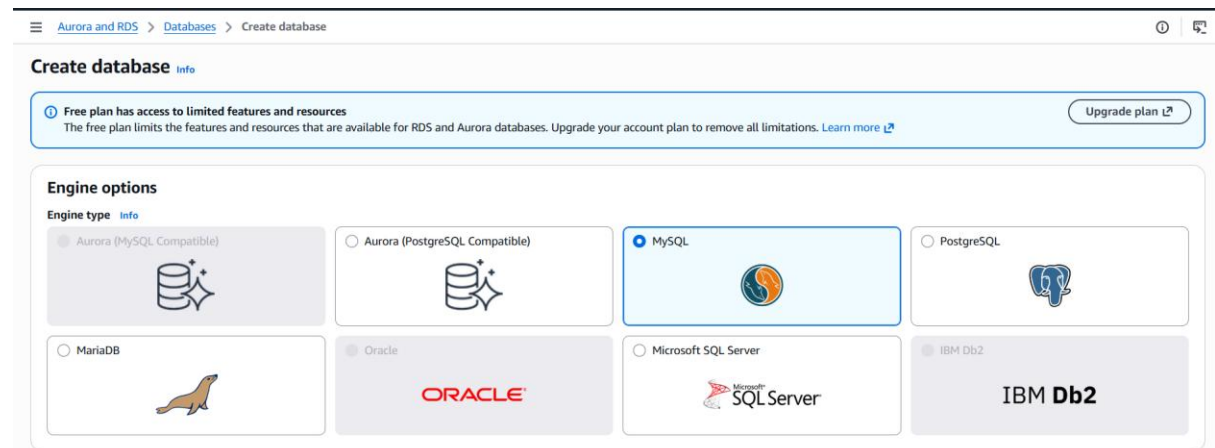
Select:

- ☒ **Standard create**

Step 4: Engine Options

Select:

- **Engine type:** MySQL
- **Engine version:** Leave the default (latest supported version).



Step 5: Templates

Choose:

- **Free tier** (for practice)

Step 6: Settings

Enter:

- **DB instance identifier:** mysql-rds
- **Master username:** admin
- **Master password:** Admin@12345
- **Confirm password:** Admin@12345

Step 7: Instance Configuration

Choose:

- **DB instance class:** db.t3.micro (or db.t4g.micro if it's the Free Tier option shown in your region)

Step 8: Storage

- **Storage type:** General Purpose SSD (gp3)
- **Allocated storage:** 20 GB (or the default)

Step 9: Connectivity

Configure:

- **VPC:** Default VPC (or the VPC where your EC2 instance is running)
- **Public access:** Yes (for practice)
- **VPC security group:** Create a new one or choose an existing one.
- **Availability Zone:** No preference (or leave the default)

Step 10: Database Authentication

Leave the default:

- **Password authentication**

Step 11: Additional Configuration

Database name (optional):

mysqldb

Leave the remaining settings as default unless your lab specifies otherwise.

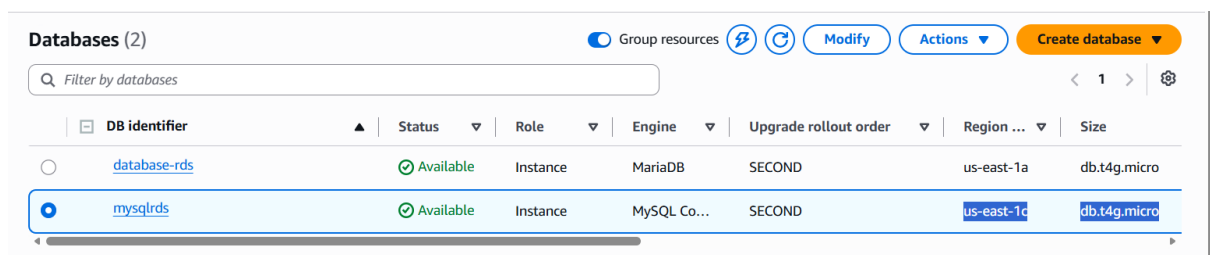
Step 12: Create Database

Click:

Create database

Wait until the status changes to:

Available



Databases (2)							
☐ Group resources Modify Actions Create database							
Filter by databases							
☐	DB identifier	Status	Role	Engine	Upgrade rollout order	Region ...	Size
☐	database-rds	Available	Instance	MariaDB	SECOND	us-east-1a	db.t4g.micro
☒	mysqldb	Available	Instance	MySQL Co...	SECOND	us-east-1c	db.t4g.micro

8. Configure Multi-AZ.

9. Take backup of DB and restore the DB.

Part A: Take a Manual Backup (Snapshot)

Step 1: Open RDS

- Login to **AWS Console**.
- Search for **RDS**.
- Click **Databases**.

Step 2: Select Your Database

Select your RDS database.

Example:

mysql-rds

Step 3: Take a Snapshot

Click:

Actions → Take snapshot

Step 4: Enter Snapshot Name

Example:

mysql-rds-snapshot

Click:

Take Snapshot

The screenshot shows the 'Take DB Snapshot' dialog box in the AWS console. The breadcrumb trail at the top reads 'Aurora and RDS > Snapshots > Take snapshot'. The dialog has a title 'Take DB Snapshot' and a 'Preferences' section. Under 'Preferences', there is a note: 'To take a DB Snapshot, choose a database and name your DB Snapshot.' Below this, the 'DB instance' section shows a dropdown menu with 'mysqlrds' selected. The 'Snapshot name' section has a text input field containing 'mysql-rds-snapshot'. A detailed note below the input field states: 'Snapshot Identifier is case insensitive, but stored as all lower-case, as in "mysnapshot". Cannot be null, empty, or blank. Must contain from 1 to 255 alphanumeric characters or hyphens. First character must be a letter. Cannot end with a hyphen or contain two consecutive hyphens.' At the bottom right of the dialog are two buttons: 'Cancel' and 'Take snapshot'.

☰ Aurora and RDS > Snapshots > Take snapshot ⓘ 📄

Take DB Snapshot

Preferences
To take a DB Snapshot, choose a database and name your DB Snapshot.

DB instance
DB instance identifier. This is the unique key that identifies a DB instance.

mysqlrds ▼

Snapshot name
Identifier for the DB Snapshot.

mysql-rds-snapshot

Snapshot Identifier is case insensitive, but stored as all lower-case, as in "mysnapshot". Cannot be null, empty, or blank. Must contain from 1 to 255 alphanumeric characters or hyphens. First character must be a letter. Cannot end with a hyphen or contain two consecutive hyphens.

Cancel Take snapshot

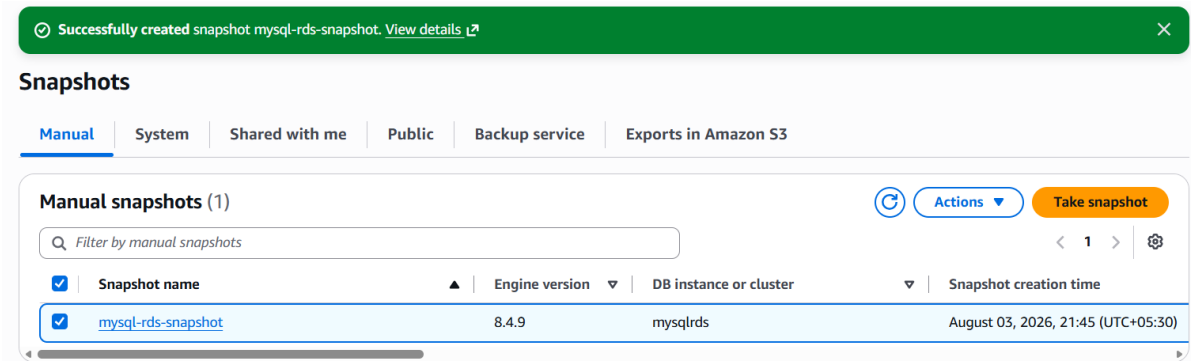
Step 5: Wait

Go to:

RDS → Snapshots

Wait until the snapshot status becomes:

Available



Part B: Restore the Database

Step 1: Open Snapshots

Go to:

RDS → Snapshots

Step 2: Select the Snapshot

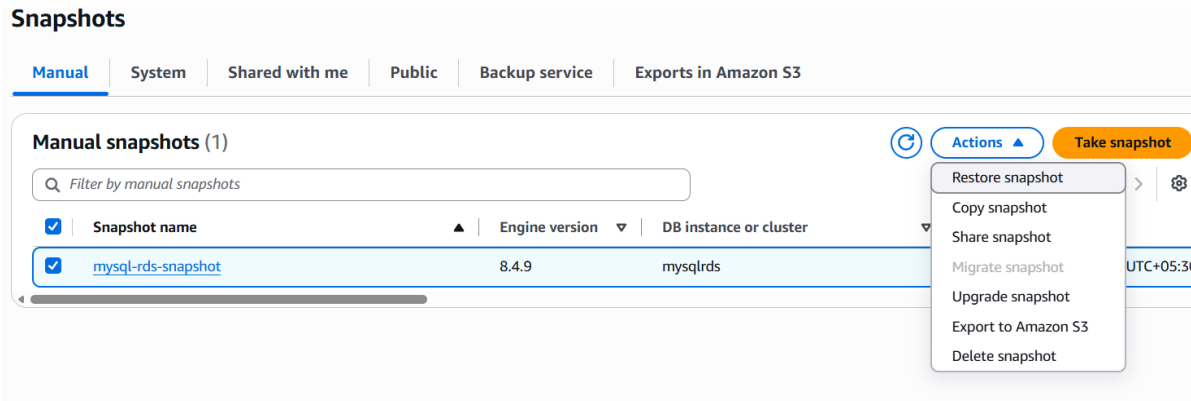
Select:

mysql-rds-snapshot

Step 3: Click Restore Snapshot

Click:

Actions → Restore Snapshot



Step 4: Configure the New Database

Enter:

DB Instance Identifier

Example:

mysql-rds-restore

Choose:

- Engine: MySQL (same as snapshot)
- DB Class: db.t3.micro (or the free-tier eligible class available)
- VPC: Same as the original database
- Security Group: Same as the original database

Leave the remaining settings as default.

Step 5: Restore

Click:

Restore DB Instance

Step 6: Wait

The new database status will change from:

Creating

to

Available

The screenshot shows the AWS Management Console interface. At the top, a green notification banner states: "Snapshot restored successfully. Restoring mysql-rds-snapshot to mysql-rds-snapshot was successful." Below this, the "Databases (2)" section is visible. It includes a search bar with the placeholder "Filter by databases", a "Group resources" toggle, and buttons for "Modify", "Actions", and "Create database". A table lists the database instances. The table has columns: DB identifier, Status, Role, Engine, Upgrade rollout order, Region, and Size. One instance is listed: "mysql-rds-snapshot" with a status of "Available", role of "Instance", engine of "MySQL Co...", rollout order of "SECOND", region of "us-east-1a", and size of "db.t3.micro".

DB identifier	Status	Role	Engine	Upgrade rollout order	Region	Size
mysql-rds-snapshot	Available	Instance	MySQL Co...	SECOND	us-east-1a	db.t3.micro

Part C: Verify the Restored Database

Open:

RDS → Databases

You should now see:

mysql-rds

and

mysql-rds-restore

The restored database has a **new endpoint**.

mysql-rds-snapshot.c2jeigsek2ps.us-east-1.rds.amazonaws.com

Snapshot restored successfully
Restoring mysql-rds-snapshot to mysql-rds-snapshot was successful.

Databases (2)

Group resources

Modify

Actions

Create database

Filter by databases

< 1 >

DB identifier	Status	Role	Engine	Upgrade rollout order	Region ...	Size
mysql-rds-snapshot	Available	Instance	MySQL Co...	SECOND	us-east-1a	db.t3.micro
mysqlrds	Available	Instance	MySQL Co...	SECOND	us-east-1c	db.t4g.micro

```
[root@ip-172-31-3-14 ec2-user]# mysql -h mysql-rds-snapshot.c2jeigsek2ps.us-east-1.rds.amazonaws.com -P 3306 -u admin1 -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 17
Server version: 8.4.9 Source distribution

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> SHOW DATABASES;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sys |
+-----+
4 rows in set (0.02 sec)
```

10. Create read replica.

Step 1: Open RDS

1. Login to the **AWS Console**.
2. Search for **RDS**.
3. Click **Databases**.

Step 2: Select the Primary Database

Click your primary RDS instance.

Example:

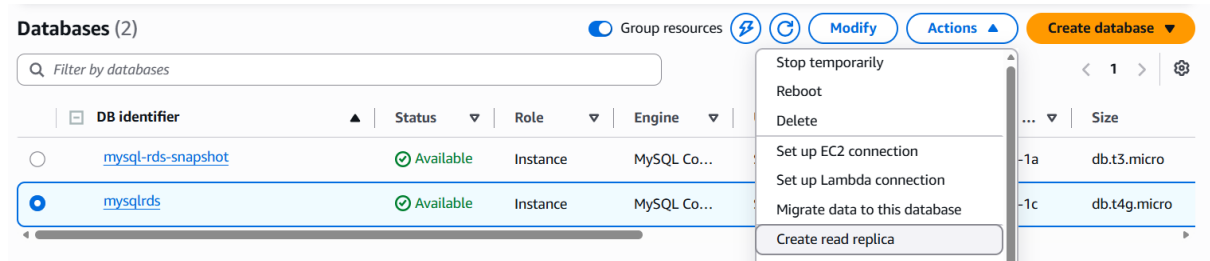
mysql-rds

Make sure its status is **Available**.

Step 3: Create Read Replica

Click:

Actions → Create read replica



Step 4: Configure the Read Replica

DB Instance Identifier

Example:

mysql-read-replica

DB Instance Class

Choose:

db.t3.micro

(or the free-tier eligible class available in your region)

Region

Choose:

- **Same Region** (recommended for practice)

or

- **Another Region** (Cross-Region Replica)

Availability Zone

Leave as:

No Preference

Storage

Leave the default settings.

Step 5: Create Read Replica

Click:

Create Read Replica

Step 6: Wait

The status will be:

Creating

After a few minutes:

Available

Step 7: Verify

Go to:

RDS → Databases

You should now see:

mysql-rds (Primary)

mysql-read-replica (Read Replica)

Step 8: Connect to the Read Replica

Copy the **Read Replica Endpoint**.

Connect from EC2:

```
mysql -h <READ-REPLICA-ENDPOINT> -P 3306 -u admin -p
```

Check the data:

```
SHOW DATABASES;
```

```
USE mysql;
```

```
SELECT * FROM table1;
```

You should see the same data as the primary database.

Successfully created replica [mysql-read-replica](#) in US East (N. Virginia)

Databases (2)

Group resources

Modify

Actions

Create database

Filter by databases

< 1 >

DB identifier	Status	Role	Engine	Upgrade rollout order	Region ...	Size
mysqlrds	Available	Primary	MySQL Co...	SECOND	us-east-1c	db.t4g.micro
mysql-read-replica	Available	Replica	MySQL Co...	SECOND	us-east-1a	db.t4g.micro
